

## DATA 202 — Quiz 2

Week 4 — Covering Week 3: Cleaning, Grouping & Plots

Name: \_\_\_\_\_

Date: \_\_\_\_\_

### The Dataset: People Experiencing Homelessness

`homeless` contains 100 simulated records of people experiencing homelessness, cleaned the same way as in Monday's class: standardized city names, shelter status, and education levels.

**Columns:** `id`, `years_homeless`, `family_size`, `monthly_support_usd` (numerical) · `name`, `city`, `shelter_status`, `education_level` (categorical/text)

**First few rows (already cleaned):**

| id | name   | city          | shelter_status    | years_homeless | family_size | monthly_support_usd | education_level |
|----|--------|---------------|-------------------|----------------|-------------|---------------------|-----------------|
| 2  | Joseph | Boston        | street            | 10             | 2           | 366                 | None            |
| 8  | Lucas  | San Francisco | shelter temporary | 8              | 2           | 25                  | Primary         |
| 21 | Lucas  | Boston        | unsheltered       | 9              | 5           | 84                  | None            |
| 39 | Anna   | Los Angeles   | shelter           | 12             | 5           | 556                 | Primary         |
| 59 | David  | San Francisco | shelter           | 11             | 2           | 264                 | Secondary       |

### Question 1 (SLO 03A)

The raw (uncleaned) `shelter_status` column also has spacing variants like "`shelter , pending`", "`shelter,pending`", and "`shelter , pending`" — the same status, just spaced differently. Complete the code below (which already lowercases the text) to standardize all of these into "`shelter pending`".

```
homeless["shelter_status"] = (  
    homeless["shelter_status"]  
    .str.strip()  
    .str.lower()  
    .str.replace(r"_____", "shelter pending", regex=True)  
)
```

### Question 2 (SLO 03B)

Complete the code below to compute, for each `education_level`, both the number of people and the average `years_homeless`, using named aggregation.

Then answer: if you used "`median`" instead of "`mean`" for `avg_years`, would the result be more or less sensitive to one extremely high outlier in `years_homeless`?

```
result = homeless.groupby("_____").agg(  
    n_people=("id", "count"),  
    avg_years=("_____", "_____"),  
)
```

### Question 3 (SLO 03C)

For each question below about the `homeless` data, name the best plot type (Histogram / Scatter / Line / Bar) and which column(s) you'd map to x (and y, if relevant).

| Question                                                                                        | Plot Type | Column(s) |
|-------------------------------------------------------------------------------------------------|-----------|-----------|
| What's the distribution of <code>years_homeless</code> across all people?                       |           |           |
| How do <code>family_size</code> and <code>monthly_support_usd</code> relate to each other?      |           |           |
| How does average <code>family_size</code> change as <code>years_homeless</code> increases?      |           |           |
| How does average <code>years_homeless</code> compare across <code>shelter_status</code> values? |           |           |

### Question 4 (SLO 02A) — Review from Quiz 1

For each command below, identify whether it **Accesses**, **Adds**, or **Deletes** data, and whether it operates on a **Row** or a **Column**.

| # | Command                                                                                   | Access / Add / Delete? | Row(s) or Column? |
|---|-------------------------------------------------------------------------------------------|------------------------|-------------------|
| 1 | <code>homeless["years_homeless"]</code>                                                   |                        |                   |
| 2 | <code>homeless["long_term"] = homeless["years_homeless"] &gt; 10</code>                   |                        |                   |
| 3 | <code>homeless = pd.concat([homeless, pd.DataFrame([new_row])], ignore_index=True)</code> |                        |                   |
| 4 | <code>homeless = homeless[homeless["city"] != "Boston"]</code>                            |                        |                   |
| 5 | <code>homeless = homeless.drop(columns=["family_size"])</code>                            |                        |                   |

### Question 5 (SLO 02B) — Review from Quiz 1

Complete the code below to select all people in “**San Francisco**” with `monthly_support_usd` below **100**, sorted by `monthly_support_usd` from lowest to highest.

*Remember: the filter condition still goes inside the square brackets and must reference `homeless` again, e.g. `homeless[homeless["Column"] ...]`.*

```
result = homeless[
    _____
]
result = result.sort_values("_____", ascending=_____)
print(len(result))
```

### Question 6 (SLO 02C) — Review from Quiz 1

Available visual channels: `x`, `y`, `color`, `size`, `symbol`, `facet_col`.

As on Quiz 1: since this is a paper quiz, `color` isn't something you can sketch in pencil — the code below uses `symbol` instead, mapping a categorical column to different *marker shapes* (e.g. `×`, `○`, `△`) rather than colors.

For the code below, fill in the table: for each visual channel it uses, name the column mapped to it and say whether that column is Numerical or Categorical.

```
px.scatter(homeless, x="family_size", y="monthly_support_usd",
           symbol="education_level", size="years_homeless",
           title="Family Size vs. Monthly Support")
```

| Visual Channel | Column Used | Data Type (Numerical / Categorical) |
|----------------|-------------|-------------------------------------|
| x-axis         |             |                                     |
| y-axis         |             |                                     |
| symbol         |             |                                     |
| size           |             |                                     |

In the space below, sketch what you expect this chart to look like. Label your axes. You don't need exact data — just show the general pattern (point cloud, marker shapes (`×`/`○`/`△`) for groups, varying point sizes).